Inflation Basket Tracker

Automated Price Monitoring | ML Forecasting | Streamlit Dashboard

Playwright | SQLite | Random Forest | GitHub Actions | Plotly

5	7	~76%	Daily	6
Items Tracked	Day Forecast	Model R2	Auto Run	Modules

1. Project Overview

The Inflation Basket Tracker is an end-to-end automated data pipeline that monitors daily prices of 5 essential grocery items (Milk, Eggs, Bread, Rice, Oil) scraped from an e-commerce website using Playwright. Prices are stored in a SQLite database, processed with Pandas, and fed into a Random Forest model to forecast the weekly grocery basket cost. The entire pipeline runs automatically every morning via GitHub Actions, and results are displayed on an interactive Streamlit dashboard.

Problem Statement

Official government inflation metrics (CPI) are lagging indicators and broad averages that don't reflect the real-time cost of living for specific households. This project solves that by monitoring a personalised 'basket' of goods daily, providing immediate visibility into price fluctuations and forecasting short-term grocery expenses -- a personalised, real-time inflation monitor.

30-Second Elevator Pitch

An automated pipeline that scrapes daily grocery prices using Playwright, stores them in a database, uses a Random Forest ML model to forecast the weekly grocery bill, and displays everything on a custom Streamlit dashboard -- running automatically every morning via GitHub Actions.

2. Tech Stack

Layer	Technology	Why Chosen
Web Scraping	Playwright (Python)	Handles JS-heavy SPAs; faster than Selenium
Database	SQLite	Zero config, file-based, native Python support
Data Processing	Pandas & NumPy	Efficient time-series aggregation & cleaning
Machine Learning	Scikit-learn (RF)	Non-linear, robust, minimal hyperparameter tuning
Dashboard	Streamlit & Plotly	Interactive charts, rapid development

Automation	GitHub Actions (cron)	Daily pipeline trigger at 01:30 UTC
Version Control	Git	Code + dataset versioning in one repo
Language	Python 3.10+	Primary language for all layers

3. Project Architecture & File Structure

Module / File	Purpose
scraper/scrape_prices.py	Playwright headless browser -- extracts item prices
database/db.py	SQLite schema setup, insert & query operations
processing/	Feature engineering: date_ordinal, day_of_week, is_weekend
ml/train_model.py	Trains RandomForestRegressor on latest DB data
ml/predict.py	Generates 7-day basket cost forecast CSV
dashboard/app.py	Streamlit dashboard -- visualises history + predictions
pipeline/run_pipeline.py	Orchestrates all stages in sequence
.github/workflows/	GitHub Actions YAML -- daily cron job definition
config/	Settings (items list, URLs, thresholds)
data/prices.db	SQLite database file (versioned in Git)

End-to-End Data Flow

```
GitHub Actions (cron: 01:30 UTC daily)
        |
scraper/scrape_prices.py  (Playwright)  ->  Raw prices
        |
database/db.py  ->  INSERT into price_data table (SQLite)
        |
processing/  ->  Feature Engineering
   Add: date_ordinal, day_of_week, day_of_year, is_weekend
        |
ml/train_model.py  ->  Retrain RandomForestRegressor
        |
ml/predict.py  ->  7-day forecast CSV
        |
GitHub Actions commits updated DB + CSV back to repo
        |
dashboard/app.py (Streamlit)  ->  http://localhost:8501
```

4. Database Design

Table: price_data

Column	Type	Description
id	INTEGER PK	Auto-incrementing unique identifier
date	TEXT	ISO 8601 date string (YYYY-MM-DD)
item_name	TEXT	Product name (Milk, Eggs, Bread, Rice, Oil)
price	REAL	Item price scraped from website
website	TEXT	Source domain (e.g. BigBasket)

* Why SQLite: Zero configuration, file-based (version-controlled via Git), native Python support -- ideal for single-user analytics at this scale.
* Data Integrity: Schema enforced via CREATE TABLE with column types.

* Duplicates managed by application logic (could be a UNIQUE constraint on date+item).
* Adding 'website' column required a migration (drop/recreate table) -- a key lesson in schema evolution.

5. Machine Learning Details

Model: Random Forest Regressor (scikit-learn)

Property	Detail
Model Type	RandomForestRegressor (ensemble of decision trees)
Target Variable	Total daily basket cost (sum of all 5 items)
Features	date_ordinal, day_of_week, day_of_year, is_weekend
Accuracy (R2)	~0.76 on synthetic data
Prediction Output	7-day forward forecast as a CSV file
Retraining	Daily -- pipeline retrains on the full latest dataset

Why Random Forest over Linear Regression?

* Linear Regression assumes a straight-line relationship -- rarely true for volatile grocery prices.
* Random Forest captures non-linear patterns and interactions (e.g. weekend price hikes).
* Robust to outliers (voting across many trees reduces overfitting).
* Requires minimal hyperparameter tuning compared to Gradient Boosting.
* Limitation: Does not extrapolate well outside training range (tree-based models plateau at the max/min seen in training).

Feature Engineering

Feature	Type	Captures
date_ordinal	Integer (days since epoch)	Long-term price trend
day_of_week	0-6 integer	Weekly pricing cycles
day_of_year	1-365 integer	Seasonal patterns
is_weekend	0 or 1 binary flag	Weekend price surges

6. Web Scraping Details

* Tool: Playwright (Python sync API) -- chosen because BigBasket and similar e-commerce sites are Single Page Applications (SPAs) that load via JavaScript. requests/BeautifulSoup only gets the initial HTML shell.
* Pattern: Launches a headless Chromium browser, navigates to each product page, waits for the price element to appear, then extracts and returns the value.
* Challenge: Anti-bot measures (WAF/Cloudflare) frequently block headless browsers with 'Access Denied' errors.
* Error Handling: Every navigation and extraction wrapped in try-except; if a selector fails or times out, None is returned and the pipeline logs the error but continues without crashing.
* Explicit waits used for async-loaded price elements.

7. Automation -- GitHub Actions

```
# .github/workflows/daily_pipeline.yml
on:
  schedule:
    - cron: '30 1 * * *'   # Daily at 01:30 UTC = 07:00 AM IST

jobs:
```

```
run-pipeline:
  runs-on: ubuntu-latest
  steps:
    - Checkout code
    - Set up Python
    - pip install -r requirements.txt
    - playwright install chromium
    - python scraper/scrape_prices.py
    - python ml/train_model.py
    - python ml/predict.py
    - git commit & push (updated DB + predictions CSV)
```

8. Dashboard Features

Key Metric Cards (Top of Dashboard)

* Latest Basket Cost -- today's total grocery spend
* Daily Inflation Rate (%) -- day-over-day percentage change
* Forecasted Cost (7 days) -- model's prediction

Charts

Chart	Type	What It Shows
Historical Price Trends	Multi-line chart	Per-item price over time
Total Basket Cost	Line chart	Historical vs. 7-day predicted cost
Daily Inflation Rate	Bar chart	Day-over-day % fluctuations

Dashboard reads directly from the SQLite DB and predictions CSV. It refreshes automatically whenever files are updated by the pipeline (e.g. after a git pull following an overnight GitHub Actions run).

9. Setup & Running

```
# 1. Navigate to project
cd d:\Project\inflation-basket-tracker\inflation-basket

# 2. Install dependencies
pip install -r requirements.txt
playwright install chromium

# 3. Run the full pipeline manually
python pipeline/run_pipeline.py

# 4. Launch dashboard
streamlit run dashboard/app.py
#  --> http://localhost:8501
```

requirements.txt

```
pandas
numpy
scikit-learn
streamlit
plotly
playwright
joblib
```

10. Challenges & Solutions

Challenge	Solution / Workaround
Anti-bot / WAF blocking Playwright	Experimented with custom headers; proxies/stealth plugin as future fix
Dynamic/async price loading	Used Playwright explicit waits for CSS selectors
Cold-start (too little data for ML)	Generated synthetic mock data to test pipeline robustness
Schema evolution (adding 'website' column)	Drop and recreate table migration strategy
Scraper failure cascades to dashboard	Error logging + pipeline continues on partial failure

11. Limitations

* Data Reliability: Dependent on website DOM structure -- UI updates break the scraper.
* Scalability: SQLite locks during writes -- not suitable for high-concurrency environments.
* Model Simplicity: Features like 'Day of Week' miss complex economic drivers (supply chain shocks, fuel prices).
* Local Execution: Dashboard runs locally -- cloud deployment needs DB persistence solution.
* Extrapolation: Tree-based RF model plateaus outside training min/max range.

12. Future Improvements

* Robust Scraping: Rotating proxies + User-Agent rotation to bypass bot detection.
* Advanced ML: Facebook Prophet or ARIMA for better seasonality + confidence intervals.
* Cloud Deployment: Streamlit Cloud or AWS; PostgreSQL/Supabase for the DB.

* Alerting: Email/Slack notification if price spikes >X% or scraping fails.
* Async Scraper: asyncio + Playwright async API to scale to 1000+ items efficiently.
* Data Quality Layer: Reject prices that deviate >50% from the rolling average.

13. Resume Bullet Points

* Engineered an end-to-end automated data pipeline extracting daily pricing data for 5+ SKUs using Python and Playwright.
* Developed a Random Forest forecasting model (scikit-learn) achieving ~76% R2 accuracy to predict weekly grocery basket costs.
* Architected a serverless system using SQLite and GitHub Actions to automate ETL jobs, reducing manual tracking effort by 100%.
* Built an interactive Streamlit dashboard visualizing real-time inflation metrics and price trends for data-driven personal finance.

14. Key Interview Q&A

Q: Explain your project in one sentence.

A: An automated pipeline that scrapes daily grocery prices to visualize real-time personal inflation trends and forecast future basket costs.

Q: Why Random Forest over Linear Regression?

A: Linear Regression assumes a straight-line relationship which rarely exists in volatile prices. Random Forest captures non-linear patterns and feature interactions (e.g. weekend price hikes) better.

Q: Why SQLite instead of MySQL/PostgreSQL?

A: For a single-user analytics project where data volume is small (<1 GB), SQLite creates zero overhead and simplifies CI/CD since the DB is just a file in Git.

Q: How does the automation work?

A: A GitHub Actions workflow triggers daily. It spins up a runner, installs dependencies, executes the pipeline scripts sequentially, and commits the new data back to the repo automatically.

Q: Why Playwright and not Selenium?

A: Playwright is faster, more reliable for modern SPAs, and has better native handling of waiting for network events and selectors.

Q: How do you handle scraper failures?

A: try-except blocks log errors without crashing. In production, I would add a retry mechanism with exponential backoff and alert notifications.

Q: How is this different from a Kaggle project?

A: This is an end-to-end system: I built data ingestion (scraper), storage (DB), automation (CI/CD), ML forecasting, and a user interface (dashboard) -- not just a model trained on a static CSV.

Q: What did you learn from this?

A: The complexity of maintaining a data pipeline -- a small scraper failure cascades to the model and dashboard, emphasizing the need for robust error handling.